

OpenStore

GitHub AI Explorer — Technical & Product Documentation

Version	1.0 — September 2026
Author / Operator	Jose Quevedo
Repository	https://github.com/Josequevedov08/OpenStore
Live app	https://app-repositorio-github-one.vercel.app
Stack	React + Vite + Tailwind (frontend) · FastAPI (backend)
Hosting	Vercel (frontend) · Render (backend, free tier)
License	See repository LICENSE

1. What is OpenStore

OpenStore is a search engine for public GitHub repositories that replaces a technical README with an AI-written, plain-language commercial pitch. A visitor types what they need in plain words ("a CRM", "a todo list app", a repository URL, or a GitHub username), and the app returns a set of app-store-style cards: a catchy title, a short pitch explaining what the project does and why it matters, its key features, what you need to run it, and — when the README makes it possible — a one-click installer for the visitor's own machine.

There is no sign-up, no account, and no paywall. The project runs entirely on free-tier infrastructure (GitHub's public API, a free-tier AI provider, free hosting on Vercel and Render), which is why several of the engineering decisions documented here exist specifically to make a free deployment behave reliably under real, shared usage limits.

2. Architecture

The system is a classic two-tier web app: a static single-page frontend and a stateless backend API, with no database. All state that needs to persist lives either in the browser's own `localStorage` (per-visitor, never sent anywhere) or in short-lived, in-memory structures on the backend (cache, rate limiter, analytics counters) that reset whenever the process restarts.

2.1 Frontend

React 19 + Vite, styled with Tailwind CSS v4 (CSS-first configuration, no `tailwind.config.js`). Deployed as a static build on Vercel. All theming — light/dark — is driven by CSS custom properties defined once in `index.css` and consumed everywhere through Tailwind's arbitrary-value syntax (e.g. `bg-[var(--surface)]`), so no component hardcodes a color.

2.2 Backend

Python 3 + FastAPI, deployed on Render's free tier (the service sleeps after inactivity — the first request after a period of inactivity can take 30–50 seconds just to wake it up; the frontend proactively pings `/health` on load to start this early). The backend orchestrates four phases per search: (1) find candidate repositories via GitHub's Search API, (2) fetch each repository's metadata and raw README in parallel, (3) send each README to an AI model to produce a structured commercial pitch, and (4) assemble the final response, sanitizing anything that will later be used to build an installer script.

2.3 Data flow of a single search

- The browser sends the raw query text, the selected language, the requested page, and the sort order to `POST /api/buscar-soluciones`.
- The backend checks a rate limiter for the caller's IP, then an in-memory cache keyed by (query, language, page, sort).
- On a cache miss, it queries GitHub's Search API for up to 12 repositories per page, then downloads each repository's README in parallel.
- Each README is sent to the configured AI provider with a strict prompt: translate everything into the requested language and return one specific JSON shape (title, pitch, features, requirements, tech stack, package manager, start command).

- The AI calls run with a concurrency limit and a hard per-call timeout, so a single slow or hung call never blocks the whole search; any repo whose AI call fails falls back to an honest, clearly-labeled placeholder built from GitHub's raw metadata.
- The response is cached (only if at least one card came from a real AI call) and returned to the browser.

3. Core features

- **Natural-language search** across all public GitHub repositories, plus direct repo URLs, GitHub profile URLs, and the "@username" shorthand to browse a person's or organization's repositories.
- **AI-written commercial pitch per repository**: a hook, what it is, how it works, and a call to action — instead of raw technical documentation.
- **Bilingual by design**: English by default, Spanish as a toggle. The AI automatically translates from whatever language the original README is written in.
- **Sort by top stars or recently updated**, with a quality floor (a minimum star count) automatically applied to "recently updated" so it doesn't surface obscure, barely-starred repositories that merely happen to match the text.
- **"Load more" instead of numbered pages**: results accumulate on one continuous screen.
- **The Smart Installer**: a downloadable script (.bat for Windows, one OS-detecting .sh for macOS/Linux) that checks for Git and the right runtime, offers to install missing tools via the OS's own package manager, clones the repository, installs its dependencies, and runs it — all in one double-click. See Section 5 for how this is kept safe.

4. Advanced features

- **Real light/dark theme**, saved per browser and applied before the page's first paint (no flash of the wrong theme).
- **Favorites**: bookmark any result and browse them in a dedicated view.
- **Compare**: pick up to three repositories and see them side-by-side — stars, forks, open issues, language, license, and detected tech stack.
- **Client-side filters**: narrow the current result set by language, license, or "has an installer".
- **Install history**: every successful installer download is logged locally with its title, repository, platform, and date.
- **Accessibility**: a visible keyboard-focus outline throughout the interface.
- **Installable as a PWA**: a web app manifest and a minimal service worker let visitors install OpenStore like a native app on desktop or mobile. Only the app shell works offline — search always needs a live connection to GitHub and the AI provider.
- All of the above (theme, favorites, comparisons, recent searches, install history) live exclusively in the visitor's own browser (localStorage) — never sent to the backend, never tied to an identity, because there is none to tie it to.

5. Security design

The single most delicate part of this system is the Smart Installer: it turns text written by a third party (a repository's README) and read by an AI model into a script that a real person will double-click and run on their own machine. That pipeline is treated as fully untrusted end to end.

5.1 Command sanitization

The AI is never allowed to write an arbitrary install command as free text. It may only choose a package manager from a closed, hardcoded set (npm, yarn, pnpm, pip, poetry, cargo, go, bundler, composer, dotnet, docker, or "none") — each mapped, server-side, to one literal, pre-written command. The separate "start command" field the AI extracts from the README (e.g. "npm run dev") passes through a strict allow/deny-list validator before it can

ever reach a generated script: it rejects command chaining (&&, ;, |), redirects, network-fetch tools (curl, wget), privilege escalation (sudo), encoded/obfuscated PowerShell, and anything not built from a short allow-list of known-safe binaries. This is covered by an automated test suite (16+ cases) with deliberately adversarial payloads (e.g. "... && rm -rf /").

5.2 What sanitization cannot do

No amount of command sanitization can audit the actual source code of a third-party repository. The installer only controls how the project is fetched and started — once it runs, it is exactly as trustworthy as its own author's code. This limitation is surfaced explicitly to visitors in the FAQ, the Terms, and the visual manual, with the same one-line advice repeated everywhere: only install repositories from authors you trust.

5.3 Abuse and quota protection

- Per-IP rate limiting on the search endpoint (configurable, default 10 requests / 5 minutes) protects the shared free-tier AI and GitHub quota from being exhausted by one user or script.
- An in-memory, 15-minute search cache means an identical query is served instantly without spending any AI or GitHub quota.
- A secondary AI provider can be configured as an automatic fallback: if the primary provider fails for a specific repository (quota, timeout, malformed response), that one repository is retried with the fallback before giving up and showing an honest "analysis pending" placeholder.
- A hard per-call timeout on every AI request prevents one hung call from blocking an entire search.

5.4 Admin panel

A read-only GET `/api/admin/stats` endpoint exposes aggregate operational counters (total searches, cache hit rate, AI-processed vs. fallback ratio, the most common search terms, rate-limit rejections). It is disabled by default (returns 503) and only activates once the operator sets an `ADMIN_TOKEN` environment variable; every request is checked against it with a constant-time comparison. A small standalone dashboard at `/admin.html` reads this endpoint — it is not linked from the main app and requires the token to view anything.

6. Privacy and data handling

OpenStore has no user accounts and does not know who any visitor is. There is no third-party analytics service (no Google Analytics, no Meta Pixel) and no tracking cookie.

- A search's text is sent to GitHub's public Search API and to the configured AI provider, under their own respective privacy terms.
- The backend keeps only in-memory, aggregate counters for capacity planning (see 5.4) — wiped every time the process restarts, never written to disk, never tied to an IP or a person.
- Everything else that needs to persist — theme, favorites, recent searches, comparisons, install history — lives exclusively in the visitor's own browser (localStorage) and is never sent to the backend.
- An optional, currently-inactive integration can log AI-generated results to a private Google Sheet for the operator's own recordkeeping; it only activates if the operator manually configures credentials for it, and it never stores anything that identifies a visitor.

The full, current text lives in the app itself at `/privacidad.html` (Privacy Policy), `/terminos.html` (Terms & Conditions), and `/faq.html` (FAQ) — this section summarizes them; those pages are the source of truth and are updated whenever the product changes.

7. Configuration reference

All backend configuration is via environment variables (see `backend/.env.example` in the repository).

- **GITHUB_TOKEN** — strongly recommended: 60 requests/hour without one vs. 5,000/hour with a read-only token; a single search already uses roughly 13 requests.
- **AI_PROVIDER / AI_API_KEY / AI_MODEL** — which AI service to use (gemini, openai, groq, or anthropic) and its model. Required for AI-written pitches; without a key, every card falls back to a metadata-only placeholder.
- **AI_FALLBACK_PROVIDER / AI_FALLBACK_API_KEY / AI_FALLBACK_MODEL** — optional second provider retried per-repository if the primary one fails.
- **AI_CONCURRENCY / AI_CALL_TIMEOUT_SECONDS** — how many AI calls run in parallel per search, and the max wait per call before falling back.
- **SEARCH_CACHE_TTL_SECONDS** — how long an identical search is served from cache (default 900s / 15 min).
- **RATE_LIMIT_MAX_PETICIONES / RATE_LIMIT_VENTANA_SEGUNDOS** — per-IP search rate limit (default 10 requests / 300s).
- **ADMIN_TOKEN** — enables the admin stats endpoint (Section 5.4); unset by default.
- **CORS_ORIGINS** — comma-separated list of allowed frontend origins.
- Frontend: **VITE_API_URL** — the backend's base URL. Vite bakes this in at build time, not at runtime; changing it requires a redeploy, and pasting anything other than a bare URL (e.g. a Markdown link by accident) breaks every request in a way that is easy to misdiagnose as a server outage (see Section 9).

8. Testing and continuous integration

The most security-critical logic — the installer's command sanitization — has automated test coverage: valid commands are accepted, and a battery of adversarial inputs (command chaining, destructive one-liners, encoded PowerShell, network downloaders, privilege escalation, overly long strings) is confirmed rejected. The admin endpoint's authentication states (missing token, wrong token, disabled endpoint, correct token) and the analytics counters are covered as well.

A GitHub Actions workflow runs the backend's pytest suite and a frontend production build on every push and pull request against the main branch, so a change that breaks either is caught before it reaches production.

9. Known limitations and lessons learned

- **Shared free-tier quotas.** Everything runs on free tiers (GitHub API, AI provider, Render, Vercel). Heavy simultaneous use by many visitors can occasionally exhaust a shared quota; the app degrades honestly (a clearly-labeled "analysis pending" placeholder) rather than failing silently or showing fabricated data.
- **Cold starts.** Render's free tier sleeps the backend after inactivity; the first request after a period of idleness can take 30–50 seconds. The frontend proactively wakes it on page load, before the visitor even searches.
- **The installer cannot audit third-party source code.** Sanitization protects the command that starts a project, not the project's own logic once it runs (see Section 5.2).
- **A build-time environment variable is easy to corrupt silently.** A Markdown-formatted link pasted into `VITE_API_URL` instead of a bare URL once broke every single request in production, with an error message generic enough to look like a server outage rather than a configuration mistake. The fix was adding an explicit, visible URL-format check that fails loudly in the browser console at startup.

- **Renamed environment variables can leave stale, wrong-typed values behind** on a hosting dashboard. A variable renamed from a float-typed name to an int-only one left a decimal value sitting in the dashboard, which would have crashed the process on the next restart. The fix was defensive parsing that logs a warning and falls back to a default instead of raising.
- **A URL query parameter must never decide where a secret gets sent.** The admin dashboard (Section 5.4) originally read its backend URL from its own "?api=" parameter with no validation, then sent the admin token there and persisted that host to localStorage. A link built as "<real-domain>/admin.html?api=https://attacker.example" carried a genuine domain and a valid padlock — nothing in the address bar would tip off the person opening it — and the moment they submitted their token, the page's own client-side code handed it straight to the attacker, with no exploit of the backend needed. The fix was a strict allow-list (the production backend's own origin, or localhost for local development) checked against both the URL parameter and anything already in localStorage; any other value is ignored outright and never persisted. Admin or credential-bearing pages need to validate destination origins against a hardcoded allow-list — a same-domain link can still turn the page itself into the phishing delivery mechanism otherwise, and localStorage persistence extends the exposure beyond the first click. See SECURITY.md for the full writeup.

10. Getting started (local development)

- Backend: create backend/.env from backend/.env.example, fill in at least AI_API_KEY (and ideally GITHUB_TOKEN), then run "pip install -r requirements.txt" and "uvicorn main:app --reload" from the backend/ directory.
- Frontend: create frontend/.env from frontend/.env.example pointing VITE_API_URL at your local backend, then run "npm install" and "npm run dev" from the frontend/ directory.
- Tests: run "pytest" from the backend/ directory.

11. Links and contact

- Repository: <https://github.com/Josequevedov08/OpenStore>
- Live app: <https://app-repositorio-github-one.vercel.app>
- Visual manual, FAQ, Terms & Conditions, Privacy Policy: linked from the app's footer.
- Questions about the project or its data handling: reach the operator via GitHub (github.com/Josequevedov08).